

Biashara AI

Development Handbook

AI Prompt Series — State-Based with HANDOFF.md

Version 2.0 — May 2026

Overview

This handbook provides a structured series of AI prompts designed to guide a coding assistant through the end-to-end development of the Biashara AI mobile application. By following these prompts in sequence, you will build a robust, offline-first Android application leveraging on-device Gemma AI to empower African SMEs.

Each prompt integrates with a centralized HANDOFF.md file to ensure seamless continuity across sessions and agents.

Key Technical Decisions (v2.0 updates)

Area	Previous (v1.0)	Updated (v2.0)
On-device LLM runtime	Generic TFLite Interpreter	LiteRT-LM (com.google.mediatube:tasks-genai)
Model format	.tflite files bundled in APK	.litemlm / .task files — downloaded at first run
Annotation processing	KAPT	KSP (Kotlin Symbol Processing — faster)
Inventory entry	Manual text entry only	Manual entry + CameraX + ML Kit barcode scanning
Model reference	Gemma 1B/4B (TFLite)	Gemma 3 1B / Gemma 3n E2B (LiteRT-LM, INT4 quantized)
Device check	None	Capability check at launch;

Area	Previous (v1.0)	Updated (v2.0)
		graceful fallback for low-end devices
Localization	Not addressed	Prompt 2b: strings.xml + KSP + RTL support for Amharic

Initial Setup: Create HANDOFF.md

Goal: Initialize the HANDOFF.md file to track project progress and state across agent sessions.

Prompt: Create HANDOFF.md

Create a new Markdown file named HANDOFF.md in the project root with the following content:

```
# Biashara AI Project Handoff Document

## Current Project State
- Last Completed Prompt: None (Project Initialization)
- Next Expected Prompt: Prompt 1 – Project Setup and Dependencies
- Key Decisions/Notes:
  - Target Platform: Android (Kotlin)
  - Architecture: Single-Activity, MVVM, Hilt, Room, Navigation Component
  - AI Integration: On-device Gemma 3 1B via LiteRT-LM (tasks-genai)
  - Annotation Processing: KSP (not KAPT)
  - Barcode Scanning: CameraX + ML Kit
- Dependencies Added: None yet.
- Files Created/Modified: None yet.

## Instructions for Next Agent
1. Read this HANDOFF.md thoroughly.
2. Execute the Next Expected Prompt.
3. Update HANDOFF.md upon completion.
```

PHASE 1: Foundation and Architecture

Prompt 1: Project Setup and Dependencies

Goal: Create the Android project and configure all dependencies including KSP, LiteRT-LM, and ML Kit.

Next Prompt: Prompt 2: Core Application Structure and Navigation

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Create a new Android Studio project named 'BiasharaAI' with an Empty Activity template using Kotlin. Configure build.gradle (Module: app) with the following dependencies:

- Material Design components (latest stable)
- Kotlin Coroutines — core and Android-specific (latest stable)
- Android Architecture Components — ViewModel, LiveData (latest stable)
- Room Persistence Library — runtime and KSP processor (latest stable)
- Navigation Component — fragment and UI-ktx (latest stable)
- Hilt for dependency injection — Android and KSP processor (latest stable)
- LiteRT-LM / MediaPipe GenAI: com.google.mediapipe:tasks-genai (latest stable)
- CameraX: androidx.camera:camera-core, camera-camera2, camera-lifecycle, camera-view (latest stable)
- ML Kit Barcode Scanning: com.google.mlkit:barcode-scanning (latest stable)

Critical: Use KSP (Kotlin Symbol Processing), NOT KAPT, for Room and Hilt. Apply the KSP Gradle plugin (com.google.devtools.ksp) and use ksp() instead of kapt() for all annotation processors. KSP is significantly faster and is the current Google recommendation.

Provide the complete build.gradle (Module: app) and settings.gradle content.

Handoff Documentation

Last Completed	Prompt 1: Project Setup and Dependencies
Next Prompt	Prompt 2: Core Application Structure and Navigation
Key Notes	KSP configured. All dependencies added. LiteRT-LM and ML Kit included.
Files Modified	build.gradle (Module: app), settings.gradle

Prompt 2: Core Application Structure and Navigation

Goal: Establish foundational architecture — main activity, navigation graph, base components.

Next Prompt: Prompt 2b: Localization and Language Setup

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Implement the following core architectural components:

- **MainActivity.kt:** Host a NavHostFragment. Create nav_graph.xml with a starting HomeFragment destination and placeholders for InventoryListFragment, SalesFragment, InsightsFragment, SettingsFragment.
- **Bottom Navigation:** Add a BottomNavigationView to MainActivity with five items: Home, Inventory, Sales, Insights, Settings.
- **Base Components:** Create abstract BaseFragment.kt and BaseViewModel.kt. BaseViewModel provides a CoroutineScope for background tasks.
- **Hilt Setup:** Create BiasharaApp.kt annotated with @HiltAndroidApp. Create AppModule.kt as an empty @Module for application-wide dependencies.
- **HomeFragment:** Display a welcome message and placeholder cards for daily summary and low-stock alerts.

Provide code for MainActivity.kt, nav_graph.xml, bottom_nav_menu.xml, HomeFragment.kt, fragment_home.xml, BaseFragment.kt, BaseViewModel.kt, BiasharaApp.kt, AppModule.kt.

Handoff Documentation

Last Completed	Prompt 2: Core Application Structure and Navigation
Next Prompt	Prompt 2b: Localization and Language Setup
Key Notes	Single-activity architecture established. Bottom nav wired.
Files Created	MainActivity.kt, nav_graph.xml, bottom_nav_menu.xml, HomeFragment.kt, fragment_home.xml, BaseFragment.kt, BaseViewModel.kt, BiasharaApp.kt, AppModule.kt

Prompt 2b: Localization and Language Setup

Goal: Set up Android localization infrastructure for Swahili, Hausa, Yoruba, and Amharic — including RTL support.

Next Prompt: Prompt 3: Local Database with Room

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Implement the full localization infrastructure:

- **strings.xml:** Externalize ALL UI strings into res/values/strings.xml. Create locale-specific folders: res/values-sw/ (Swahili), res/values-ha/ (Hausa), res/values-yo/ (Yoruba), res/values-am/ (Amharic). Provide translations for all strings in each language.
- **RTL support:** Add android:supportsRtl="true" to the <application> tag in AndroidManifest.xml. Amharic (am) uses Ethiopic script and requires RTL layout. Ensure all layouts use start/end attributes instead of left/right.

- **Language selection:** Create a LanguageSelectionFragment that appears on first launch. Display each language in its native script. Save the selected locale to SharedPreferences and apply it using AppCompatActivity.setApplicationLocales().
- **Language persistence:** On subsequent launches, read the saved locale from SharedPreferences and apply before setContentView in MainActivity.

Provide strings.xml for English and Swahili as examples, the LanguageSelectionFragment, and updated AndroidManifest.xml.

Handoff Documentation

Last Completed	Prompt 2b: Localization and Language Setup
Next Prompt	Prompt 3: Local Database with Room
Key Notes	RTL support enabled. Language selection screen implemented. Locale persistence via SharedPreferences.
Files Created	res/values/strings.xml, res/values-sw/strings.xml, res/values-am/strings.xml, LanguageSelectionFragment.kt, fragment_language_selection.xml — updated AndroidManifest.xml

PHASE 2: Data and User Interface

Prompt 3: Local Database with Room

Goal: Implement Room database with Product entity, DAO, and barcode field.

Next Prompt: Prompt 4: Inventory Management UI

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Extend the BiasharaAI project to include a Room database. Define a Product data class as a Room @Entity with the following fields:

- id: Long — Primary key, auto-generated
- name: String — Product name
- description: String? — Nullable description
- price: Double — Selling price
- cost: Double — Cost price
- stockQuantity: Int — Current stock
- category: String? — Nullable category
- barcodeValue: String? — Barcode or QR code value (scanned or manually entered)
- imageUrl: String? — Path to product image

Create a ProductDao interface with: insertProduct, updateProduct, deleteProduct, getAllProducts(): Flow<List<Product>>, getProductById(id: Long): Flow<Product?>, getProductByBarcode(barcodeValue: String): Flow<Product?>.

Create the AppDatabase abstract class. Use Flow (not LiveData) for reactive queries — it integrates better with Kotlin Coroutines and the LiteRT-LM async inference calls. Provide Product.kt, ProductDao.kt, AppDatabase.kt.

Handoff Documentation

Last Completed	Prompt 3: Local Database with Room
Next Prompt	Prompt 4: Inventory Management UI
Key Notes	barcodeValue field added. Using Flow instead of LiveData for reactive queries.
Files Created	Product.kt, ProductDao.kt, AppDatabase.kt

Prompt 4: Inventory Management UI

Goal: Implement the inventory list screen with RecyclerView and scan-to-search capability.

Next Prompt: Prompt 4b: Barcode and QR Code Scanning

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Create InventoryListFragment.kt and fragment_inventory_list.xml:

- **Layout:** RecyclerView showing product name, stock quantity, price, and a small barcode icon if barcodeValue is set. Include a Floating Action Button (FAB) for adding new products, and a scan icon button in the toolbar to open barcode scanning.
- **ProductAdapter.kt:** RecyclerView.Adapter binding Product data to list items. Clicking an item navigates to AddEditProductFragment for editing.
- **InventoryListViewModel.kt:** Inject ProductDao via Hilt. Expose `StateFlow<List<Product>>` from `getAllProducts()`. Collect the Flow in a `viewModelScope` coroutine.
- **Fragment:** Observe StateFlow from ViewModel. Handle FAB click (navigate to AddEditProductFragment for new product). Handle scan icon click (navigate to BarcodeScannerFragment — defined in Prompt 4b).

Provide fragment_inventory_list.xml, ProductAdapter.kt, InventoryListViewModel.kt, InventoryListFragment.kt.

Handoff Documentation

Last Completed	Prompt 4: Inventory Management UI
Next Prompt	Prompt 4b: Barcode and QR Code Scanning
Key Notes	Scan icon added to toolbar. StateFlow used for reactive list.
Files Created	fragment_inventory_list.xml, ProductAdapter.kt, InventoryListViewModel.kt, InventoryListFragment.kt

Prompt 4b: Barcode and QR Code Scanning

Goal: Implement CameraX + ML Kit barcode scanning for inventory lookup and product entry — fully offline.

Next Prompt: Prompt 5: Product Entry and Editing

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Implement barcode and QR code scanning using CameraX and Google ML Kit Barcode Scanning. All detection runs on-device with no internet connection required.

Dependencies (already in build.gradle from Prompt 1): com.google.mlkit:barcode-scanning, androidx.camera:camera-core, camera-camera2, camera-lifecycle, camera-view.

- **Permissions:** Add `<uses-permission android:name="android.permission.CAMERA"/>` to AndroidManifest.xml. Request camera permission at runtime in BarcodeScannerFragment using `ActivityResultContracts.RequestPermission()`.
- **BarcodeScannerFragment.kt:** Display a live camera preview using CameraX PreviewView. Overlay a viewfinder box (semi-transparent rectangle) centered on screen

to guide the user. Support three scan modes passed as a navigation argument: SCAN_FOR_LOOKUP (search existing product), SCAN_TO_ADD (pre-fill new product form), SCAN_TO_RECORD_SALE (open POS with product).

- **BarcodeAnalyzer.kt:** Implement ImageAnalysis.Analyzer. Use BarcodeScannerOptions.Builder() configured for Barcode.FORMAT_UPC_A, FORMAT_UPC_E, FORMAT_EAN_13, FORMAT_EAN_8, FORMAT_QR_CODE. Pass frames to ML Kit via InputImage.fromMediaImage(). On detection, extract rawValue and navigate based on scan mode.
- **CameraX setup:** Use CameraController with MlKitAnalyzer to handle coordinate transformations automatically. Bind to the fragment lifecycle. Use STRATEGY_KEEP_ONLY_LATEST as the backpressure strategy.
- **Result handling:** On successful scan in SCAN_FOR_LOOKUP mode, query ProductDao.getProductByBarcode(). If found, navigate to AddEditProductFragment. If not found, show a snackbar: 'Product not found. Add it?'

Provide BarcodeScannerFragment.kt, fragment_barcode_scanner.xml, BarcodeAnalyzer.kt, and updated nav_graph.xml to include BarcodeScannerFragment with a scanMode argument.

Handoff Documentation

Last Completed	Prompt 4b: Barcode and QR Code Scanning
Next Prompt	Prompt 5: Product Entry and Editing
Key Notes	Fully offline. Three scan modes: LOOKUP, ADD, SALE. MlKitAnalyzer handles coordinate transforms.
Files Created	BarcodeScannerFragment.kt, fragment_barcode_scanner.xml, BarcodeAnalyzer.kt — updated nav_graph.xml

Prompt 5: Product Entry and Editing

Goal: Implement Add/Edit product screen with voice input and barcode field.

Next Prompt: Prompt 6: Device Capability Check and Model Download

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Create AddEditProductFragment.kt and fragment_add_edit_product.xml:

- **Layout:** EditText fields for name, description, price, cost, stock quantity, category, and barcodeValue. Include a scan icon button next to the barcodeValue field that opens BarcodeScannerFragment in SCAN_TO_ADD mode. Include a microphone icon button next to the name field for voice input. Include a Save button.
- **AddEditProductViewModel.kt:** Inject ProductDao via Hilt. Support loading existing product by ID for editing. Validate inputs (non-empty name, valid numbers). Call insertProduct or updateProduct.
- **Voice input:** Use Android's built-in SpeechRecognizer (ACTION_RECOGNIZE_SPEECH intent) as a placeholder. The on-device Gemma voice processor will replace this in Prompt 8.

- **Barcode pre-fill:** If navigated from BarcodeScannerFragment with a scanned value, pre-populate the barcodeValue field automatically.

Update nav_graph.xml to include navigation from InventoryListFragment and BarcodeScannerFragment to AddEditProductFragment, passing optional productId and barcodeValue arguments.

Handoff Documentation

Last Completed	Prompt 5: Product Entry and Editing
Next Prompt	Prompt 6: Device Capability Check and Model Download
Key Notes	Barcode field with scan shortcut. Voice input placeholder (SpeechRecognizer). Pre-fill from scanner.
Files Created	fragment_add_edit_product.xml, AddEditProductViewModel.kt, AddEditProductFragment.kt — updated nav_graph.xml

PHASE 3: On-Device AI Integration (LiteRT-LM)

IMPORTANT CONTEXT FOR ALL PHASE 3 PROMPTS: The on-device AI runtime for Biashara AI is LiteRT-LM (previously the MediaPipe LLM Inference API for Android, which is now deprecated). Models are in .litemlm or .task format — not .tflite. The dependency is `com.google.mediapipe:tasks-genai`. Models must be downloaded at runtime (first launch) and stored in internal storage — they cannot be bundled in the APK.

Prompt 6: Device Capability Check and Model Download

Goal: Check device compatibility for on-device LLM inference and implement the first-run model download flow.

Next Prompt: Prompt 7: LiteRT-LM Integration and Gemma Inference

Prompt

Before starting this prompt, read `HANDOFF.md` to understand the current project state.

Implement device capability checking and model lifecycle management:

- **DeviceCapabilityChecker.kt:** Create a utility class that checks: available RAM (`ActivityManager.getMemoryInfo()`), Android API level (minimum API 28 for reliable LLM inference), and available internal storage (minimum 1.5GB free). Return a `CapabilityTier` enum: `FULL_AI` (4GB+ RAM), `PARTIAL_AI` (3GB RAM), `RULES_BASED` (below threshold).
- **ModelDownloadManager.kt:** On first launch when `CapabilityTier` is `FULL_AI` or `PARTIAL_AI`, prompt the user to download the Gemma model (~600MB). Show a download progress screen with a clear warning about data usage. Download the model to `getFilesDir()/models/`. Store the download state in `SharedPreferences`.
- **SettingsFragment update:** Add an 'AI Model' section showing model status (Downloaded / Not Downloaded / Downloading), storage used, and a Re-download button.
- **AppModule.kt update:** Provide `CapabilityTier` as a Hilt-injected singleton so all `ViewModels` can check it before calling AI features.

Note: Do not initialize the LiteRT-LM `LlmInference` object in this prompt. That is done in Prompt 7. This prompt only handles the download and device check.

Handoff Documentation

Last Completed	Prompt 6: Device Capability Check and Model Download
Next Prompt	Prompt 7: LiteRT-LM Integration and Gemma Inference
Key Notes	<code>CapabilityTier</code> injected via Hilt. Model downloaded to <code>getFilesDir()/models/</code> . Download warned users about data usage.
Files Created	<code>DeviceCapabilityChecker.kt</code> , <code>ModelDownloadManager.kt</code> — updated <code>SettingsFragment.kt</code> , <code>AppModule.kt</code>

Prompt 7: LiteRT-LM Integration and Gemma Inference

Goal: Initialize LiteRT-LM, create a GemmaService singleton, and implement demand forecasting.

Next Prompt: Prompt 8: Local Language Voice Input

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Initialize the LiteRT-LM runtime and implement on-device demand forecasting:

- **GemmaService.kt:** Create a singleton (injected via Hilt `@Singleton` scope) that initializes `LlmInference` using `LlmInferenceOptions.builder()`. Set the model path to the downloaded model file in `getFilesDir()/models/`. Initialize lazily and only when `CapabilityTier` is `FULL_AI` or `PARTIAL_AI`.

```
// Correct LiteRT-LM initialization pattern:
val options = LlmInferenceOptions.builder()
    .setModelPath(modelPath)
    .setMaxTokens(1024)
    .setPreferredBackend(LlmInferenceOptions.Backend.GPU)
    .build()
val llmInference = LlmInference.createFromOptions(context, options)
```

- **DemandForecaster.kt:** Inject `GemmaService`. Create a method `predictDemand(productName: String, salesHistory: List<Int>): String`. Build a concise text prompt from the product name and sales history (e.g., 'Sales for [product] over last 7 days: [data]. Predict next 3 days demand in the format: Day1: X, Day2: Y, Day3: Z.'). Call `llmInference.generateResponse(prompt)` and parse the output.
- **Integration:** Update `InventoryListViewModel` to call `DemandForecaster` for products with sufficient sales history (minimum 7 data points). Display the forecast as a small badge on the product list item.

Important: `LlmInference.generateResponse()` is synchronous and CPU/GPU-intensive. Always call it from a background coroutine (`Dispatchers.IO`) — never on the main thread.

Handoff Documentation

Last Completed	Prompt 7: LiteRT-LM Integration and Gemma Inference
Next Prompt	Prompt 8: Local Language Voice Input
Key Notes	<code>GemmaService</code> is a Hilt singleton. Inference always on <code>Dispatchers.IO</code> . Demand forecast shown as badge on inventory list.
Files Created	<code>GemmaService.kt</code> , <code>DemandForecaster.kt</code> — updated <code>InventoryListViewModel.kt</code> , <code>InventoryListFragment.kt</code>

Prompt 8: Local Language Voice Input

Goal: Replace the `SpeechRecognizer` placeholder with Gemma-powered on-device voice transcription for local languages.

Next Prompt: Prompt 9: Financial Insights Generator**Prompt**

Before starting this prompt, read HANDOFF.md to understand the current project state.

Note: Gemma 3n (E2B/E4B) supports audio input natively via the LiteRT-LM multimodal API. Use this for local-language voice transcription when running on FULL_AI tier devices. For PARTIAL_AI devices, retain the Android SpeechRecognizer fallback.

- **Permissions:** Ensure RECORD_AUDIO permission is declared in AndroidManifest.xml and requested at runtime.
- **AudioCaptureHelper.kt:** Create a utility class that captures microphone audio using AudioRecord, producing a mono PCM ByteBuffer suitable for model input.
- **VoicelInputProcessor.kt:** Inject GemmaService and DeviceCapabilityChecker. On FULL_AI devices, use LlmInference with an audio prompt (Gemma 3n multimodal): pass the audio buffer and the prompt 'Transcribe this voice input in [language].'. On PARTIAL_AI/RULES_BASED devices, fall back to Android's built-in SpeechRecognizer.
- **Integration:** Replace the SpeechRecognizer placeholder in AddEditProductFragment with VoicelInputProcessor. Show a recording indicator while audio is being captured. After transcription, populate the active EditText field.

Handoff Documentation

Last Completed	Prompt 8: Local Language Voice Input
Next Prompt	Prompt 9: Financial Insights Generator
Key Notes	Gemma 3n multimodal audio used on FULL_AI. SpeechRecognizer fallback on PARTIAL_AI. Language-aware transcription prompt.
Files Created	AudioCaptureHelper.kt, VoicelInputProcessor.kt — updated AddEditProductFragment.kt

Prompt 9: Financial Insights Generator

Goal: Implement on-device cash flow analysis using Gemma, with localised narrative output.

Next Prompt: Prompt 10: Testing Suite

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Implement the cash flow insights feature:

- **Transaction entity and DAO:** Create a Transaction Room entity with: id, type (INCOME/EXPENSE), amount, description, date. Create TransactionDao with insertTransaction, getAllTransactions(): Flow<List<Transaction>>, getTransactionsByPeriod(start: Long, end: Long): Flow<List<Transaction>>.
- **CashFlowAnalyzer.kt:** Inject GemmaService and TransactionRepository. Implement generateInsights(transactions: List<Transaction>, language: String): String. Build a

structured prompt summarizing total income, total expenses, and top 3 expense categories. Include the active language so Gemma responds in the user's language.

- **Prompt template:**

```
You are a friendly business advisor for a small African business.  
Respond in [language]. Keep your response under 150 words.  
Financial summary for [period]:  
Total income: [amount]. Total expenses: [amount].  
Top expenses: [category1]: [amount], [category2]: [amount].  
Provide 2-3 specific, actionable insights.
```

- **CashFlowInsightsFragment + ViewModel:** Display a chart of income vs. expenses (use MPAndroidChart or a simple custom view). Below the chart, show the Gemma-generated narrative. Show a loading spinner while inference runs. Show a 'Refresh' button to regenerate.

Handoff Documentation

Last Completed	Prompt 9: Financial Insights Generator
Next Prompt	Prompt 10: Testing Suite
Key Notes	Gemma responds in user's selected language. Insights capped at 150 words for readability on small screens.
Files Created	Transaction.kt, TransactionDao.kt, TransactionRepository.kt, CashFlowAnalyzer.kt, CashFlowInsightsFragment.kt, CashFlowInsightsViewModel.kt

PHASE 4: Testing and Deployment

Prompt 10: Testing Suite

Goal: Implement unit and integration tests for the data layer, AI inference, and barcode scanning.

Next Prompt: Prompt 11: Final Review and Optimization

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Implement a testing suite covering critical paths:

- **Room DAO tests:** Use an in-memory RoomDatabase to test ProductDao and TransactionDao — insertProduct, getAllProducts, getProductByBarcode, getTransactionsByPeriod.
- **ViewModel tests:** Use turbine library to test StateFlow emissions from InventoryListViewModel and CashFlowInsightsViewModel with mock data.
- **DemandForecaster unit test:** Mock GemmaService to return a controlled response string. Verify that DemandForecaster.predictDemand() correctly parses 'Day1: 5, Day2: 8, Day3: 6' format output.
- **BarcodeAnalyzer test:** Write an instrumented test that passes a pre-generated barcode bitmap to the analyzer and verifies the correct rawValue is returned.
- **Acceptance criteria:** Room query time < 50ms for 1,000 products. Gemma inference time < 8 seconds on FULL_AI tier device. Barcode detection latency < 500ms on a real device.

Handoff Documentation

Last Completed	Prompt 10: Testing Suite
Next Prompt	Prompt 11: Final Review and Optimization
Key Notes	Performance baselines established. All DAO and ViewModel tests passing.
Files Created	ProductDaoTest.kt, InventoryListViewModelTest.kt, DemandForecasterTest.kt, BarcodeAnalyzerInstrumentedTest.kt

Prompt 11: Final Review and Optimization

Goal: Performance audit, offline validation, and cloud sync implementation.

Next Prompt: None — Project Complete

Prompt

Before starting this prompt, read HANDOFF.md to understand the current project state.

Conduct the final review and optimization pass:

- **Database optimization:** Profile all Room queries using Android Studio Database Inspector. Add @Query indexes on frequently filtered fields (barcodeValue, category). Verify all queries complete within acceptance criteria.
- **LiteRT-LM efficiency:** Verify the GemmaService singleton is initialized only once per app session. Confirm all inference calls are wrapped in Dispatchers.IO. Measure Time To First Token and decode speed using the approach documented in the Google AI Edge Gallery source.
- **Offline validation:** Enable airplane mode and exercise every user flow: onboarding, language selection, product add/edit, barcode scan, POS sale, demand forecast, cash flow insights, voice input. All flows must complete without error.
- **Cloud sync:** Implement optional sync to Firebase Firestore (encrypted) when connectivity is available. Sync is user-opt-in and configurable in Settings. Use a last-write-wins strategy for conflict resolution.
- **Rules-based fallback:** Verify that on a RULES_BASED tier device, the app loads without attempting to initialize LiteRT-LM, all AI feature buttons are disabled with a tooltip explaining device requirements, and all other features work normally.

Final Handoff Documentation

Last Completed	Prompt 11: Final Review and Optimization
Next Prompt	None — Project Complete
Performance results	Document Room query times, Gemma inference times, barcode detection latency from testing
Offline test result	Confirm pass/fail for each user flow tested in airplane mode
Sync strategy	Firebase Firestore, user-opt-in, last-write-wins
Final notes	List any known limitations or future enhancement recommendations

Appendix: Key API Reference

LiteRT-LM Initialization

```
// Dependency (build.gradle):
implementation 'com.google.mediarpipe:tasks-genai:0.10.27'

// Initialize LlmInference:
val options = LlmInferenceOptions.builder()
    .setModelPath(modelPath)    // path to downloaded .litertlm file
    .setMaxTokens(1024)
    .setPreferredBackend(LlmInferenceOptions.Backend.GPU)
    .build()
val llm = LlmInference.createFromOptions(context, options)

// Generate response (always on background thread):
val response = withContext(Dispatchers.IO) {
    llm.generateResponse(prompt)
}
```

ML Kit Barcode Scanning (CameraX)

```
// Dependencies (build.gradle):
implementation 'com.google.mlkit:barcode-scanning:17.3.0'
implementation 'androidx.camera:camera-core:1.3.4'
implementation 'androidx.camera:camera-camera2:1.3.4'
implementation 'androidx.camera:camera-lifecycle:1.3.4'
implementation 'androidx.camera:camera-view:1.3.4'

// BarcodeScanner setup:
val options = BarcodeScannerOptions.Builder()
    .setBarcodeFormats(
        Barcode.FORMAT_UPC_A, Barcode.FORMAT_UPC_E,
        Barcode.FORMAT_EAN_13, Barcode.FORMAT_EAN_8,
        Barcode.FORMAT_QR_CODE
    ).build()
val barcodeScanner = BarcodeScanning.getClient(options)

// MLKitAnalyzer with CameraController:
cameraController.setImageAnalysisAnalyzer(
    ContextCompat.getMainExecutor(this),
    MLKitAnalyzer(
        listOf(barcodeScanner),
        COORDINATE_SYSTEM_VIEW_REFERENCED,
        ContextCompat.getMainExecutor(this)
    ) { result ->
        val barcode = result?.getValue(barcodeScanner)?.firstOrNull()
        barcode?.rawValue?.let { onBarcodeDetected(it) }
    }
)
```

Official Documentation Links

- LiteRT-LM: <https://github.com/google-ai-edge/LiteRT-LM>

- LLM Inference Android guide:
https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/android
- ML Kit Barcode Scanning: <https://developers.google.com/ml-kit/vision/barcode-scanning/android>
- CameraX ML Kit Analyzer:
<https://developer.android.com/media/camera/camerax/mlkitanalyzer>
- LiteRT Community (HuggingFace): <https://huggingface.co/litert-community>
- Gemma models: <https://ai.google.dev/gemma>